DSE6211 Final Report

Tracey Zicherman

2024-06-28

Executive Summary

There is a clear strategic benefit for operators of hotel properties to have a better picture of which bookings are likely to be canceled. Such a picture can be developed more easily than ever using sophisticated methods of modern machine learning, which can make use of information related to the booking, and other customer features, to produce predictions of which bookings are likely to be canceled in advance.

This project developed such a model for ABC Hotels using a popular method of machine learning known as deep neural networks. While neural network models have been developed for decades, the introduction of so-called deep models, with many hidden layers, led to dramatic improvements in the early 21st century. These models are being increasingly employed in industry, to dramatic effect, across many aspects of business and other sectors.

After obtaining booking data from ABC hotels, the data were subjected to thorough quality checks, and explored to identify patterns and create new features which could prove useful for predictive modeling. Standard best practices were observed at every step during the data cleaning and preparation and feature engineering phases, as well as during the preprocessing phase, when the data was prepared for the machine learning modeling phase.

During the modeling phases, extensive prototyping and experimentation was used to gain a basic familiarity with the data, the predictive modeling problem, and the performance of deep neural network models in this context. Many different simple model architectures were investigated, trained, and adjustments made to improve performance.

From the prototyping phase, two models were selected for training, and their predictions subjected to further investigation and analysis. Following standard best practices, a portion of the training data was held out to simulate real unseen data, and standard evaluation metrics and visualizations were developed for both models on this holdout data, and used to estimate model performance in production on data derived from actual future bookings.

The models were then compared and contrasted, and from this, the better of the two models selected, with an estimated 83.6% accuracy for predicting future booking cancellations. Finally, recommendations were made, along with suggestions for future research and improvements.

Approach and Data

Overall Approach

The predictive modeling aspect of this project proceeded in 6 main phases.

1. ***Data Cleaning*** The data was carefully inspected and was subjected to a number of checks and cleaning steps, to ensure its quality for further use.
2. ***Feature Engineering*** – New derivative features were created from preexisting features, including a binary feature identifying high retention customers, and some preexisting features were transformed.
3. ***Exploratory Data Analysis*** – Exploratory Data Analysis: Extensive investigation of all features was conducted both univariate analysis (including visualizations of all single feature distributions) and bivariate analysis (including some pairwise distribution visualizations).
4. ***Preprocessing*** – Data was pre-processed to prepare for predictive modeling with machine learning.
5. ***Modeling*** – Dense neural network predictive classification models were prototyped, leading to two model architectures which were trained and their predictions obtained.
6. ***Model Evaluation and Selection*** - The two models were evaluated, compared and contrasted, and the best model selected, as evaluated by several standard criteria for classification models.

Data Set and Features

The original dataset consisted of 36328 observations with 17 features.

List of Booking Features
Booking_ID
no_of_adults
no_of_children

List of Booking Features
no_of_weekend_nights
no_of_week_nights
type_of_meal_plan
required_car_parking_space
room_type_reserved
lead_time
arrival_date
market_segment_type
repeated_guest
no_of_previous_cancellations
no_of_previous_bookings_not_canceled
avg_price_per_room no_of_special_requests
booking_status

Data Cleaning

During the cleaning phase, the data was subjected to standard checks and corrections, including

- Eliminating duplicate observations and features.
- Removing features incompatible with or problematic for predictive modeling.
- Replacing problematic and missing values.

- Enforcing correct data types.

The resulting data set was then used to engineer new derivative features to improve predictive modeling efforts.

Feature Engineering

The following new derivative features were created

- `arrival year`, `arrival_month`, `arrival_day` - These features were extracted from the booking date arrival_date, in order to expose trend, cyclical and seasonal information to modeling.
- `total_guests` - Total number of guests (`no_of_adults` + `no_of_children`)
- `total_nights` - Total number of booking nights (`no_weekends` + `no_weekdays`)
- `booking_value` - Overall booking value (`avg_price_per_room` x `total_nights`)

Preprocessing

We performed the following preprocessing steps:

- `step_other` - this pools infrequently occurring values of nominal (categorical) features into another category called "other"
- `step_dummy` - this one-hot encodes all categorical features.
- `step_center` - this centers all features (which are all numeric by this stage in the processing pipeline) by subtracting them from their mean.
- `step_scale` - this scales all features by dividing them by their standard deviation.

We perform a standard 80-20 train-test split, since the dataset is relatively small with roughly 36000 observations. After feature engineering, splitting, and preprocessing, the train and sets had the following dimensions (where the first is the number of observations, the second the number of features).

```
Train set dimensions: 28990 19

Test set dimensions: 7248 19
```

Machine Learning Methods

Classification Task

The target (dependent feature) is `booking_status` which indicates whether the booking was canceled or not.

The aim of the predictive modeling task was supervised classification, and thus the positive class was `canceled` and the negative class as `not_canceled`.

Here is a table of the distribution of canceled versus non-canceled bookings, that is, the distribution of the target feature:

booking_status	Count	Percentage
canceled	11878	33%
not_canceled	24360	67%

Data Splitting

Once the data had been fully cleaned and preprocessed, it was randomly shuffled. This step corrected sharp fluctuations in accuracy on validation data between training epochs, discovered during model training.

Next the data was subjected to a standard 80-20 train-test split, reserving the test data as holdout data for the final evaluation step.

Prototyping

The predictive classification machine learning algorithm employed was known as "multilayer perceptron", or in more modern terms, a "deep neural network".

Several different architectures were tried out in the prototyping phase. Different numbers of hidden layers were tested, and within each layer, different numbers of nodes were tested. Descending numbers of nodes, starting with numerous nodes in the first layer, decreasing to fewer nodes in the last layer, seemed to give good results.

For activation functions, following common practice, the "rectified linear unit" [ReLU] was used for the hidden layers. Since the learning task is binary classification, a sigmoid activation was chosen for the last layer.

The loss function was binary cross-entropy, the default choice for binary classification. Since the classes were balanced, we chose to optimize for the accuracy metric, using the Adam optimizer, a common choice used in machine learning today.

The maximum number of training epochs at 200 to get a clearer picture of overfitting for prototype models

Model Architectures

After considerable experimentation during the prototyping phase, two models were selected for training and evaluation. Both were given deep architectures with many hidden layers.

To decrease the potential of overfitting, the following additional hidden layers were added after each processing (dense) layer.

- ***Batch Normalization***: These layers standardize the outputs of each hidden layer before sending it to the next layer. This helps correct for a number of known issues in deep learning model fitting, for example, the vanishing gradient problem. Primarily the use of batch normalization in this project was to help regularize, i.e. reduce overfitting.
- ***Dropout***: These layers randomly drop a set proportion of the nodes of the processing layer. This also helps reduce overfitting. Following best practice for the architectures used in this project, a dropout proportion of 0.5 was used for each layer

.

Here are summaries of the model 1 and 2 architectures (in order).

Model: "functional"

Layer (type)	Output Shape	Param #	Train…
input_layer (InputLayer)	(None, 23)	0	-
layer1 (Dense)	(None, 128)	3,072	Y
batch_norm1 (BatchNormalization)	(None, 128)	512	Y
dropout1 (Dropout)	(None, 128)	0	-
layer2 (Dense)	(None, 64)	8,256	Y
batch_norm2 (BatchNormalization)	(None, 64)	256	Y
dropout2 (Dropout)	(None, 64)	0	-
layer3 (Dense)	(None, 32)	2,080	Y
batch_norm3 (BatchNormalization)	(None, 32)	128	Y
dropout3 (Dropout)	(None, 32)	0	-
layer4 (Dense)	(None, 1)	33	Y

Total params: 14,337 (56.00 KB)
Trainable params: 13,889 (54.25 KB)
Non-trainable params: 448 (1.75 KB)

Layer (type)	Output Shape	Param #	Train…
input_layer (InputLayer)	(None, 23)	0	-
layer1_1 (Dense)	(None, 512)	12,288	Y
batch_norm1_1 (BatchNormalization)	(None, 512)	2,048	Y
dropout1_2 (Dropout)	(None, 512)	0	-
layer2_1 (Dense)	(None, 256)	131,328	Y
batch_norm2_1 (BatchNormalization)	(None, 256)	1,024	Y
dropout2_1 (Dropout)	(None, 256)	0	-
layer3_1 (Dense)	(None, 128)	32,896	Y
batch_norm3_1 (BatchNormalization)	(None, 128)	512	Y
dropout3_1 (Dropout)	(None, 128)	0	-
layer4_1 (Dense)	(None, 64)	8,256	Y
batch_norm4_1 (BatchNormalization)	(None, 64)	256	Y
dropout4_1 (Dropout)	(None, 64)	0	-
layer5_1 (Dense)	(None, 32)	2,080	Y
batch_norm5_1 (BatchNormalization)	(None, 32)	128	Y
dropout5_1 (Dropout)	(None, 32)	0	-
layer6 (Dense)	(None, 1)	33	Y

Total params: 190,849 (745.50 KB)
Trainable params: 188,865 (737.75 KB)
Non-trainable params: 1,984 (7.75 KB)

The additional regularization steps improved the performance of the two deeper models compared to the shallower models explored during prototyping.

Given the aforementioned tendency of both of these deep models to overfit, an early stopping criterion was used, which stopped training if no improvement was seen in 10 epochs.

Note that model 1 and model 2 have similar architectures, with the same Dense > Batch Normalization > Dropout blocks repeating, and number of nodes beginning with a high factor of two, and decreasing by factors of 2 from beginning to end.

Model 2 is essentially Model 1 with two additional Dense > Batch Normalization > Dropout blocks added at the beginning, with the number of nodes in each added layer increasing by a factor of two.

Model Training

Model 1 was able to train for a larger number of epochs than Model 2 before improvement plateaued (25 vs. 10).

The training and validation accuracy and loss curves revealed that for Model 1, the training and validation curves continued to narrow over the training epochs. For Model 2, overfit the same gap narrowing was observed for the loss curves, while the gap for the accuracy curves grew during the final training epochs. Overall, this indicated that Model 2 was overfitting more than Model 1.

Findings

The results of evaluating both Models 1 and 2 on the holdout test data confirmed the hypothesis suggested during training, that Model 2 had more of a problem with overfitting.

Results

The two models were compared on a standard selection of four classifier metrics evaluated on the test data. Model 1 had better values for all four metrics.

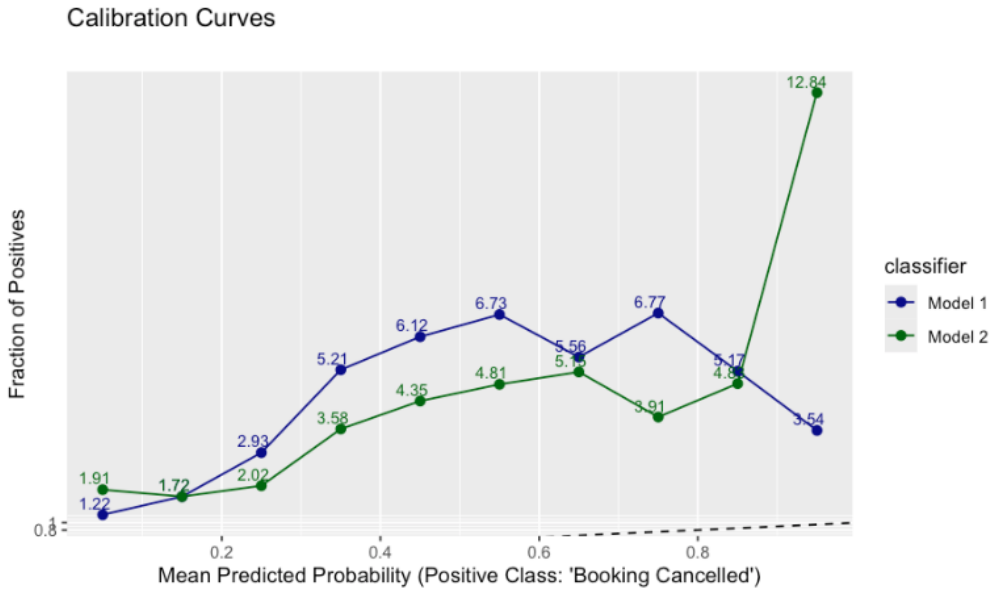
The calibration curves for both models were less than ideal, landing above the reference curve for both. This indicates both models tended to be overconfident, in that the calibration curves revealed that the predicted probabilities are generally lower than the true probabilities. Overall, Model 2 showed predicted probabilities closer to true probabilities, suggesting it was more confident in prediction canceled bookings, although a sharp deviation from the reference curve at the right end signaled potential issues.

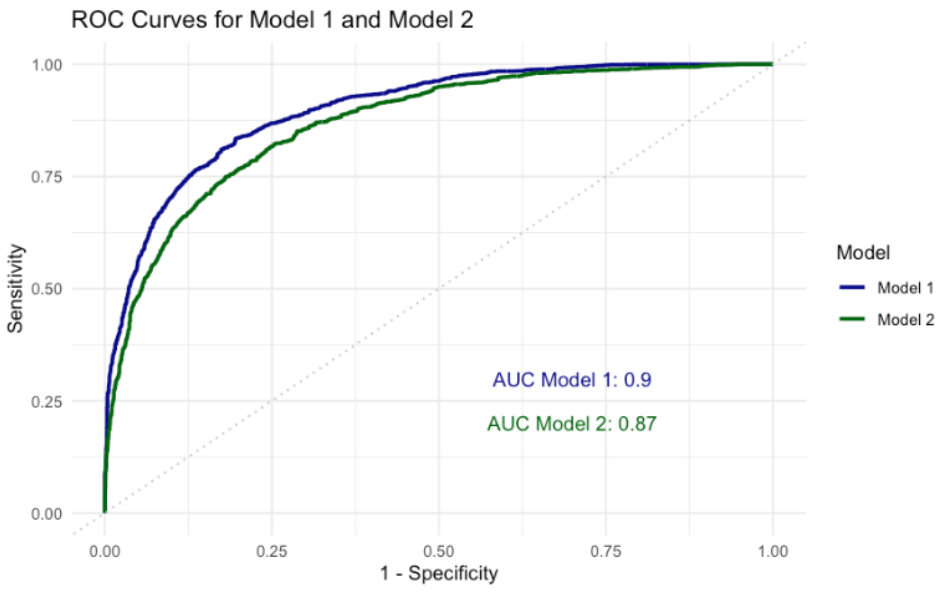
The ROC curves and AUC scores provided a clearer picture, with Model 1 showing a better curve and higher AUC than Model 2 (0.9 versus 0.87). Overall it is noted that the F1 and

AUC scores for both models, which are good measures of overall classifier performance, were relatively good.

Taking all evaluations into consideration however, Model 1 emerged as the clear choice.

Metric	Model 1	Model 2
Accuracy	0.836	0.806
Precision	0.869	0.846
Recall	0.890	0.870
F1 Score	0.873	0.858





Conclusions

Recommendations and Further Steps.

Overall, the predictive model performance with respect to predicting canceled bookings may prove insight into bookings and have a role to play in informing strategic decision making. While an accuracy of 83.6% in predicting canceled bookings falls short of perfection, it significantly outperforms chance and demonstrates substantial predictive capability. The use potential depends on the strategic needs of the company.

Recalling that the calibration curves for the selected model, Model 1 showed under confidence, it seems likely that it will tend to erroneously predict “false negatives”, that is, it will tend to fail to predict a canceled booking in advance. This means that the use of the predictions in practice will tend to lead to an overestimate of the number of bookings which will not be canceled. Reliance on these predictions could thus inform a conservative strategy, in the sense that properties would tend to be under-booked, but there would likely be fewer double bookings. This could potentially harm profits but improve customer experience,

If the opposite were true, and the model would tend to over predict cancelations, and the company were to use this prediction to say, open the booking to other customers, properties may end up overbooked. This might put the company in a worse position from a customer experience perspective, although it could reduce the number of underbooked properties, and thus have a better impact on profits.

Given the above discussion about the under-confidence of the selected predictive model, it is further recommended that if it is put into production, robust model performance data collection and monitoring systems are implemented. Controlled experiments can be designed with these systems, which could test hypotheses regarding the effect of and under confident model on company operations, and help guide future predictive modeling extensions to the current project.

Finally, as with any predictive modeling project, there is always room for improvement. For thoroughness, we mention a few possible extensions here.

- ***Better Features.*** A predictive model is only as good as the data it is trained on. Possible improvements include better feature engineering and feature selection. More extensive data analysis, including statistical analysis, could better inform this.
- ***Better Tuning.*** Hyperparameter tuning could be implemented to provide a more systematic approach to experimentation. A rapid and more automated experimentation pipeline, including tracking and logging systems could help with this.
- ***Better Models.*** Consider exploring more advanced architectures, such as deep neural networks with residual connections or recurrent/LTSM models, which can effectively capture time-dependent patterns in booking data and lead to better performing models